PROJECT REPORT

PneumoFL

Privacy-Preserving Pneumonia Detection via Federated Learning
What it is, how it was built, and what went wrong along the way

Course	B.Tech BCSE497J — Project I, SCOPE
Team	Uddhav Sethi (23BKT0092), Chirag Gadhyan (23BDS0265), Ishaan S Shrivastav (23BCI0130)
Faculty guide	Dr. Adaline Suji
Alignment	UN Sustainable Development Goal 3 — Good Health & Well-Being
Repository	UddhavSethi/privacy-preserving-medical-diagnosis
Report date	31 August 2026

1 What This Project Is, and What's Actually New About It

PneumoFL trains a chest X-ray pneumonia classifier across three simulated hospitals that never share a single patient image with each other or with a central server. Instead, each hospital trains locally and sends only a small, noised, cryptographically masked update of model weights — the images themselves never leave the hospital's own storage.

1.1 The problem it addresses

Three separate, well-known problems compound in medical imaging AI, and most published work only tackles one at a time:

- **Data cannot be centralized.** Pooling patient X-rays from multiple hospitals onto one server for training is usually illegal, unethical, or contractually impossible. Most academic pneumonia-detection papers sidestep this by simply using one public, already-pooled dataset.
- **Federated updates are not automatically safe.** Even when raw images stay local, the model updates a hospital sends can still leak information about its training data (membership inference, gradient inversion). Treating "federated" as synonymous with "private" is a common but incorrect assumption.
- **Black-box predictions aren't clinically actionable.** A prediction with no explanation and no confidence signal gives a clinician nothing to act on or challenge.

1.2 The system

PneumoFL layers six mechanisms on top of a DenseNet121 classifier, each protecting or explaining a different part of the pipeline:

Layer	Protects / provides	Mechanism
Federated Learning	Raw patient images stay local	Flower — hospitals train locally, server only aggregates
Differential Privacy	Information inside an update	Opacus DP-SGD — per-sample clipping + calibrated Gaussian noise, formal (ε, δ) accounting
Secure Aggregation	An individual hospital's update, even from the server itself	Flower SecAgg+ — cryptographic masking, server only ever recovers the sum
TLS + client authentication	Messages in transit; impersonation resistance	gRPC + mutual TLS, per-hospital certificates
Explainability	Clinical trust in individual predictions	Grad-CAM heatmaps over the frozen backbone's final conv layer
Uncertainty estimation	Deferral of low-confidence cases to a human	Monte Carlo Dropout, fixed-coverage deferral policy

1.3 What's actually novel here — checked against real published work, not assumed

It would be easy to claim "nobody has combined all of these before" without checking. That claim was tested against the current literature rather than taken on faith. The closest published comparator found is **FedIHRAS** (Biomedicines, 2026) — a federated radiological-analysis framework that also combines differential privacy, secure aggregation, and Grad-CAM explainability. It is a real, close precedent, and it should be named as one rather than talked around. It differs from PneumoFL in scope: FedIHRAS uses a ResNet-50 backbone and adds anatomical segmentation and automated SNOMED-CT-coded report generation, but — based on its published description — does not appear to include an uncertainty-estimation/deferral layer, TLS-level client authentication as a named component, or a systematic ablation quantifying what each privacy layer individually costs.

Several other papers were found combining *pairs* of these mechanisms well — federated learning with differential privacy for chest X-ray/COVID classification, and separate surveys covering federated uncertainty quantification in medical imaging as its own research thread. A few papers do run ablations over DP parameters specifically (clipping norms, noise multipliers) and report privacy–utility curves.

The fair, literature-grounded claim is therefore narrower than "first ever" and more specific: **the particular six-layer combination (FL + DP + SecAgg + TLS/client-auth + Grad-CAM + MC-Dropout deferral), implemented together as one running pipeline, with an explicit ablation ladder quantifying the accuracy/calibration/communication cost each layer adds on top of the others**, is uncommon in the literature surveyed. Individual pairs of these mechanisms are well studied; assembling and *measuring* all six together, rather than asserting they compose safely, is the part this project actually contributes. This is consistent with how the project's own governing document frames its contribution: integration and measurement, not new algorithms — DenseNet121, DP-SGD, Secure Aggregation, Grad-CAM and MC Dropout are each individually well-studied; the ablation table quantifying their combined cost is the deliverable.

> **The most persuasive single number the project produces** is the federated model outperforming any individual hospital's locally-trained model — the actual value proposition of federated learning — measured directly rather than assumed, alongside a full epsilon sweep showing the privacy/accuracy tradeoff, and (added in this session) a quantified finding that the frozen-backbone architecture required for that privacy stack measurably degrades Grad-CAM explanation quality, especially on the cases the model gets wrong.

Sources consulted: FedIHRAS — Kollias et al., PMC13023735 / doi:10.3390/biomedicines14030713; DP+FL COVID chest X-ray, PMC11193384; federated pediatric pneumonia classification, Scientific Reports (2024); privacy-preserving FL + uncertainty quantification review, Radiology: AI and arXiv:2409.16340; FL+DP medical imaging with parameter ablations, Scientific Reports (2022).

2 Technology Stack, and Why Each Piece Was Chosen

Every dependency below was picked for a specific reason tied to the project's priority order — privacy, then security, then correctness, then reproducibility, then academic credibility, then explainability, then uncertainty, then maintainability, then raw accuracy — never the reverse.

Layer	Technology	Why this, specifically
Language	Python 3.11 (pinned, not 3.12)	Exact version compatibility across the whole FL/DP toolchain (Flower, Opacus) — a mismatch here is a known source of silent breakage.
Model backbone	DenseNet121, ImageNet-pretrained	CheXNet lineage — the standard, reviewer-familiar choice for chest X-ray work. Not swapped for anything newer/flashier because reviewer familiarity and comparability mattered more than novelty here.
FL framework	Flower (flwr)	The only mainstream framework offering both a simulation engine (fast sweeps) and a real deployment engine (gRPC, real processes) behind the *same* client/server code — critical for the project's "simulate for measurement, deploy for demonstration" split.
Aggregation strategy	FedAvg	The standard, best-understood baseline. FedProx/FedBN were deliberately not substituted in — introducing a second algorithmic variable would have muddied the ablation's causal story.
Differential Privacy	Opacus (DP-SGD)	The PyTorch-native library with a real formal accountant (RDP), not just noise injection — a privacy claim without an accountant is explicitly treated as unacceptable in this project.
Secure Aggregation	Flower SecAgg+	An established, peer-reviewed masking protocol. Writing custom cryptography was explicitly ruled out — a hand-rolled scheme is a near-guaranteed reviewer attack surface.
Transport security	gRPC + mutual TLS	Server-only TLS doesn't stop a fake hospital from connecting; client authentication was required to actually back the project's impersonation-resistance claim.
Preprocessing (contrast)	OpenCV (CLAHE only)	torchvision has no CLAHE implementation; used narrowly for exactly that one operation, with parameters fixed, logged, and precomputed to disk rather than left as a per-run source of drift.
Transforms/augmentation	torchvision	Correct, composable seeding for augmentation — kept separate from CLAHE specifically so random augmentation state is never entangled with a deterministic preprocessing step.
Explainability	pytorch-grad-cam	An established library instead of a hand-rolled Grad-CAM implementation, which also made comparing Grad-CAM variants close to free.

Uncertainty	Monte Carlo Dropout	Cheap, and the method the source project deck specified. Explicitly framed as a known-weak baseline, not the strongest possible choice — stronger methods are named as future work rather than silently substituted in.
Config management	Hydra / OmegaConf	Every run fully specified by config, no hand-edited constants between runs — a reproducibility requirement, not a convenience.
Experiment tracking	MLflow (local, self-hosted)	Chosen deliberately over a cloud tracking service — a project whose whole thesis is data sovereignty should not ship its own telemetry to a third party.
Containerization	Docker Compose	Makes the multi-hospital deployment demonstration reproducible off the original machine.
Dataset acquisition	kaggle, pydicom, Pillow	RSNA is DICOM (needs pydicom); Kermany is JPEG (needs Pillow); Kaggle's API automates checksum-verified acquisition of both.
Metrics	scikit-learn	AUROC as the primary metric (not raw accuracy) — mandatory on this imbalanced medical dataset.

Deliberately excluded

TensorFlow / TF-Federated / TF-Privacy, NVFlare, OpenFL, PySyft, CrypTen, Weights & Biases or any cloud tracking service, blockchain components, Vision Transformers or medical foundation models, and Kubernetes were all considered and explicitly ruled out — several for the same reason (a second, redundant framework for something Flower/Opacus already does), and cloud tracking specifically because it would contradict the project's own data-sovereignty premise.

3 Everything That Went Wrong, and How It Was Fixed

This project's own working culture treats "found via a real run, not by inspection" as worth stating explicitly wherever it applies — a recurring pattern below. Issues are grouped by when they surfaced: the original 24-stage build (Stages 0–23 and the OPT-1–5 research extensions), and this session's debugging and repair work.

PART A — DURING THE ORIGINAL BUILD (STAGES 0–23, OPT-1 THROUGH OPT-5)

STAGE 5 RSNA validation/test shard collapsed to one hospital

What went wrong: The RSNA shard-splitting step reused the exact same seed (1000) as Stage 4's earlier patient-level train/val/test split on the same population. Because both used a sort-then-shuffle pattern, the identical seed reproduced the identical cut points instead of an independent split — Hospital C ended up with 100% train data and zero validation/test records; Hospital B absorbed all of it instead.

Fix: Changed the shard seed to 5000, regenerated both partition files, and added a regression test so the two seeds can never silently collide again.

STAGE 8 In-place ReLU silently broke per-sample gradients

What went wrong: The classifier head used `nn.ReLU(inplace=True)`, which conflicts with Opacus's backward hooks — in-place ops break the view-tracking `GradSampleModule` needs to compute per-sample gradients for DP-SGD. Found by actually running Opacus's validator, not by reading the docs.

Fix: Switched to `inplace=False`.

STAGE 13 Six real Flower-tooling gaps, found only by running it

What went wrong: The pinned Flower version had fully moved to a new Message API, superseding the older client-function style most examples online still show. On top of that: the local simulation needs a persistent SuperLink daemon that can accumulate stale state across killed runs; the default simulated node count is 2, not 10; the packaging step's default include patterns swept in ~350MB of unrelated files, blowing a 10MB bundle limit; the fix for that (a positive allowlist) silently no-ops if nested under the wrong config table, with no error; and the simulation runtime executes from a copied, isolated directory, so any path derived from `__file__` silently resolves wrong. Separately, the local training loop seeded only its shuffle order, leaving Dropout's own random mask unseeded — "identical" reproducible runs silently diverged.

Fix: Rewrote the client/server on the Message API; documented the daemon's known failure mode (kill it, delete its state DB, retry clean — see the direct echo of this in Part B below); pinned node count explicitly; switched to a positive file allowlist; moved every path to an absolute one threaded through config; added an explicit `torch.manual_seed(seed)` alongside the local shuffle generator.

STAGE 14 **DP accountant mismatch and an overflow crash**

What went wrong: The noise calibration function assumed an RDP accountant, but a bare `PrivacyEngine()` defaults to a different one ("prv") — the calibrated noise and the reported epsilon would have been computed under two different mathematical models, silently. Separately, `get_epsilon()` overflows instead of returning infinity as noise approaches zero, crashing the no-DP-equivalent edge case. A third issue was methodological: an early test tried to verify DP clipping/noise by comparing Adam-optimized parameter positions before and after, which gave the wrong-direction result because Adam's adaptive normalization confounds a raw-gradient comparison.

Fix: Pinned the accountant explicitly to RDP everywhere; wrapped epsilon computation in a guard for the overflow case; rewrote the test to inspect Opacus's internal clip/noise mechanism directly instead of optimizer output.

STAGE 15 **Secure Aggregation's default overflowed on real hospital sizes**

What went wrong: Found live: Flower's SecAgg+ workflow defaults to `max_weight=1000.0`, far below this project's real per-hospital training counts (~13,342 for the RSNA shards) — every round silently logged an overflow warning in the weight-encoding math.

Fix: Set `max-weight=20000` explicitly, with headroom for future partitions.

STAGE 16 **A rejected client hung the test suite indefinitely**

What went wrong: A negative-path test (an unregistered node attempting to connect) hung forever — `flower-supernode` doesn't exit cleanly on an auth rejection because its sidecar process keeps stdout open, and a plain `subprocess.run(timeout=...)` can't reach a subprocess's own children.

Fix: Launched with `start_new_session=True` and killed the whole process group via `os.killpg` on timeout.

STAGE 17 **Three separate Docker deployment bugs**

What went wrong: (1) Excluding `torch` by name from the Docker build still pulled ~2GB of unrelated CUDA packages, since excluding one package doesn't prune its transitive dependencies from a CUDA-oriented lockfile. (2) A TLS certificate's SAN list didn't include the Compose service name `superlink`, so all three hospital containers hung in uninformative silent retry loops instead of a clear certificate error. (3) A registration race: all containers started simultaneously, but hospital registration happened afterward and sequentially, so a hospital's first connection could land before its key was registered — a hard crash with no retry.

Fix: Used direct pinned-CPU-version installs for the Docker build only; added the missing SAN entry; reordered startup to register every hospital before any hospital container starts.

STAGE 18 **Grad-CAM hard-crashed on ADR-1's frozen backbone**

What went wrong: `pytorch-grad-cam`'s plain `GradCAM` doesn't mark its input tensor as requiring gradients by default. For an ordinarily fully-trainable model this is invisible, since some parameter upstream already requires grad — but with the backbone entirely frozen, nothing in the forward pass required gradients at all, so no autograd graph was ever built and the backward hook silently never fired. This was a hard crash, not a subtly-wrong heatmap, caught on this stage's own first live run. A second, smaller bug: the qualitative-example script collected records source-first, so Kermany's small bucket filled the sample slice before any RSNA record was reached — the first run's saved examples were accidentally 100% one dataset.

Fix: Explicit `requires_grad_(True)` on the input tensor before calling the library; a seeded shuffle before bucketing examples.

STAGE 21 **Four bugs in the full ablation campaign**

What went wrong: (1) The feature-loading function assumed one data source per hospital, which would have silently dropped every record from the second source for Dirichlet's mixed-source synthetic clients. (2) Dirichlet partitioning's own client keys didn't match the client app's hardcoded hospital mapping. (3) No federated run had ever actually been logged to MLflow before this stage — a real gap against the project's own "a result not in MLflow does not exist" rule. (4) The campaign's actual first launch attempt corrupted itself within seconds: the runner script omitted Flower's `--stream` flag, so it submitted and returned immediately instead of blocking, and all 27 runs got dispatched to the same SuperLink almost simultaneously — caught by noticing multiple simultaneous "RUNNING" entries in MLflow. A fifth issue: MLflow's SQLite filter parser rejects parenthesized boolean grouping, and naive substring name-matching would have silently pulled DP runs into the plain-FedAvg row, since one run name is a literal prefix of another.

Fix: Grouped features per (source, split) and concatenated; remapped Dirichlet client keys to match; added optional MLflow logging to both server apps; added the missing `--stream` flag and relaunched the campaign clean; switched to flat filter clauses on logged params instead of name pattern-matching.

STAGE 23 **A stray test run contaminated the production results database**

What went wrong: The real data-integrity bug of the build: an integration smoke test invoked a real federated run but never redirected its MLflow tracking URI, so every test execution silently wrote a stray result into the actual production results database. Caught only because a regenerated chart showed `n_seeds: 6` instead of the expected 3, with a visibly diluted mean.

Fix: Root-caused the three specific stray runs by timestamp and a distinctive AUROC value that matched a 1-round test result rather than the real 20-round campaign, deleted them directly, then fixed the actual source by redirecting the test's tracking URI to an isolated temp file.

OPT-3 **A background job reported finished while still running**

What went wrong: Combining a manual `nohup cmd &` with the harness's own background-execution flag caused a spurious early completion report — the shell returned immediately because of the redundant trailing `&`, while the real ~12-minute Grad-CAM evaluation job was still executing.

Fix: Caught by checking the actual process and output directory before trusting the completion signal, then waited it out with a proper blocking poll instead of double-backgrounding.

PART B — THIS SESSION

SESSION **The live app called a real pneumonia X-ray "Normal" with high confidence**

What went wrong: Traced the full inference path end to end and ruled out the obvious suspects — checkpoint loading, class-index mapping, preprocessing, and the frontend display were all verified correct. The real cause: the app's default model (FedAvg + DP at ε=4) was severely biased toward predicting "Normal" specifically on Hospital A's data — 54.9% accuracy there, worse than the 71.9% majority-class baseline, even though its AUROC (0.825) showed the underlying features were still informative. DP-SGD's per-sample clipping and noise had miscalibrated the decision threshold, not destroyed the learned representation. This was invisible in the project's existing pooled-accuracy calibration numbers because the other two hospitals are Normal-majority in their own test sets, masking the bias.

Fix: Switched the app's default model to FedAvg without DP, which showed no such collapse (82.8% accuracy, 0.950 AUROC on the same set). No model or training code was touched — an app-layer default change only, leaving the DP-protected models available under "Advanced" and the research ablation results untouched.

SESSION **The out-of-distribution warning fired on almost every real X-ray**

What went wrong: Each hospital's anomaly detector is calibrated to flag only ~5% of its own held-out data, but the app showed its caution banner if *any single* hospital's detector disagreed with an image. Tested directly: a genuine Kermany X-ray was flagged 9 times out of 10, and a genuine RSNA X-ray 10 times out of 10 — because the three hospitals' images are naturally different from each other (different scanners, different populations), so almost every real image looked foreign to at least one of the three detectors, even though it matched its own source hospital fine.

Fix: Changed the logic from "any hospital disagrees" to "no hospital recognizes it" — only flag an image when it doesn't resemble any of the three training distributions, which is what the warning is actually supposed to mean.

SESSION **Grad-CAM was measurably looking at the wrong part of the X-ray**

What went wrong: Measured directly on 150 real RSNA pneumonia X-rays with radiologist-drawn bounding boxes: when the model's prediction was correct, Grad-CAM's hottest pixel landed inside the true opacity only 18.6% of the time (already weak); on cases the model got wrong, that dropped to 0.0%. Root cause: the project's frozen-backbone architecture (necessary for the DP/FL/hardware constraints) compresses each X-ray into a single 1,024-number summary before the classifier ever sees it, so no spatial layout survives — Grad-CAM's heatmap is a reconstruction of correlated channels, not something the classifier ever actually used spatially.

Fix: Implemented the project's own pre-approved fallback for exactly this situation — unfreezing the backbone's last dense block and swapping its BatchNorm for GroupNorm (via Opacus's own conversion utility) so it stays DP-compatible while gaining real spatial learning capacity. A bounded pilot confirmed real gains on held-out data: pooled test AUROC rose from 0.9051 to 0.9256, and — the more important number — Grad-CAM's pointing-game accuracy on the same 150 images roughly doubled when correct (23.1% → 37.5%) and recovered from 13.6% to 35.2% on cases still misclassified. The architecture change is additive and off by default; every existing checkpoint is untouched.

SESSION **The federated re-run of that fix stalled repeatedly, for three different reasons**

What went wrong: Re-training the fine-tuned architecture through genuine federated rounds (rather than the faster centralized pilot used to validate the idea) hit three distinct, real failures in sequence. First, forcefully killing an earlier stuck process left Flower's local SuperLink daemon without its execution companion process, so it silently accepted new runs without ever dispatching them — no error, just permanent silence. Second, background training runs were mysteriously killed mid-flight by something outside this conversation's control (confirmed not to be the user), traced to how the harness's own background-task tracking interacts with a process that spawns a large multi-process daemon tree (Flower's SuperLink plus Ray's cluster). Third, after fixing both of those, a new evaluation step added for best-checkpoint selection stalled for nearly two hours; inspecting Ray's own internal worker logs directly (rather than guessing again) showed a single task reported "active" for ~140 minutes inside a GPU-training actor process — strong evidence that Flower's simulation engine reuses the same GPU-bound actor pool for server-side evaluation immediately after client training, and that reuse hangs on a second heavy CUDA workload.

Fix: Restarted the SuperLink daemon fully rather than partially; fully detached later runs from the harness's process tracking (`setsid` + `nohup` + `disown`) so they survive independently; and restructured the evaluation step to do no GPU work at all inside Flower's managed context — it now just saves each round's weights cheaply, with the actual accuracy/validation scoring moved to run afterward, once, in the plain process outside Ray entirely. Notably, this project's own Stage 13 notes (Part A) had already documented the SuperLink daemon's stale-state failure mode as a known issue — a fact this session rediscovered the hard way before finding it already written down.

SESSION **The checkpoint-selection logic would have silently kept a worse model**

What went wrong: Caught by the user, not found independently: the first version of the federated fine-tuning re-run saved whichever round happened to run last, regardless of whether a middle round had actually scored better — the same behavior the project's own canonical federated app already has, inherited without questioning it.

Fix: Every round's checkpoint is now scored against a held-out pooled validation set (never the test set, to avoid contaminating the final evaluation), and only the best-scoring round is kept. Verified with a direct test before relying on it: a deliberately destroyed round (weights perturbed until its AUROC collapsed to 0.5, chance level) was confirmed *not* to overwrite an earlier, genuinely better checkpoint.